

=====

=====

=====

MARKDOWN / SECTION

Nur — Lightweight Rare-Earth-Free Permanent-Magnet Screening

Goal: Materials Project → RE-free/stability/magnetism pre-screen → NovoMag/NEMAD/literature cross-match → unit standardization → lightweight hard-magnet descriptors → physical filters → Pareto optimization → sensitivity/robustness → literature-check shortlist.

Scientific safeguards

- Materials Project magnetization is used **only as a broad pre-screen**.
- Final intrinsic magnetization is taken from cross-matched magnetic-property data ($J_s = \mu_0 M_s$) or curated literature.
- Cross-match priority is **MP-ID > formula+space group > unique-formula-only**.
- NEMAD Curie temperature is used only as a **formula-level supplement** unless phase identity is independently verified.
- Anisotropy/hardness evidence is required before treating a material as a permanent-magnet candidate.
- Calculated $(BH)_{max}$ is labelled **ideal intrinsic energy product**, not measured practical energy product.
- No subjective weighted score is used as the primary selection method.

=====

=====

MARKDOWN / SECTION

=====

=====

Part 0 — Install packages

CODE CELL

```
!pip -q install mp-api pymatgen lxml html5lib openpyxl plotly
```

```
=====
=====
=====
```

MARKDOWN / SECTION

Part 1 — Imports, constants, and reproducible screening settings

CODE CELL

```
import os, re, json, math, itertools, warnings, getpass
from pathlib import Path
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
from pymatgen.core import Composition
from mp_api.client import MPRester
```

```
warnings.filterwarnings("ignore")
```

```
OUT = Path("/content/nur_pm_outputs")
OUT.mkdir(exist_ok=True, parents=True)
```

MU0 = 4.0 * np.pi * 1e-7 # H/m

Rare-earth definition used here: Sc, Y, and lanthanides La-Lu.

```
RARE_EARTHS = {  
    "Sc", "Y", "La", "Ce", "Pr", "Nd", "Pm", "Sm", "Eu", "Gd", "Tb", "Dy",  
    "Ho", "Er", "Tm", "Yb", "Lu"  
}
```

Optional actinide exclusion for practical/materials-safety reasons.

```
ACTINIDES = {  
    "Ac", "Th", "Pa", "U", "Np", "Pu", "Am", "Cm", "Bk", "Cf", "Es", "Fm", "Md", "No", "Lr"  
}  
EXCLUDE_ACTINIDES = True
```

Broad Materials Project pre-screen.

```
MP_EHULL_BROAD_MAX = 0.20 # eV/atom  
MP_MOMENT_FU_MIN = 0.50 # mu_B/formula unit; broad nonmagnetic filter only  
MP_NUM_ELEMENTS = (2, 4) # binary to quaternary  
MP_INCLUDE_GNOME = False
```

Primary hard-magnet screen.

```
PRIMARY_EHULL_MAX = 0.10 # eV/atom  
PRIMARY_TC_MIN = 400.0 # K  
PRIMARY_JS_MIN = 0.80 # T  
PRIMARY_KAPPA_MIN = 1.00  
PRIMARY_KEFF_MIN = 0.10 # MJ/m^3
```

```
ALLOW_UNIQUE_FORMULA_ONLY_NOVOMAG = True
SHORTLIST_CONFIDENCE = {"A", "B", "C"}
SHORTLIST_N = 15
```

```
CONFIG = {
    "MP_EHULL_BROAD_MAX": MP_EHULL_BROAD_MAX,
    "MP_MOMENT_FU_MIN": MP_MOMENT_FU_MIN,
    "MP_NUM_ELEMENTS": list(MP_NUM_ELEMENTS),
    "PRIMARY_EHULL_MAX": PRIMARY_EHULL_MAX,
    "PRIMARY_TC_MIN": PRIMARY_TC_MIN,
    "PRIMARY_JS_MIN": PRIMARY_JS_MIN,
    "PRIMARY_KAPPA_MIN": PRIMARY_KAPPA_MIN,
    "PRIMARY_KEFF_MIN": PRIMARY_KEFF_MIN,
}
```

```
with open(OUT / "screening_config.json", "w") as f:
    json.dump(CONFIG, f, indent=2)
```

```
print("Output folder:", OUT)
print(json.dumps(CONFIG, indent=2))
```

=====

=====

=====

MARKDOWN / SECTION

Part 2 — Helper functions for formulas, numbers, space groups, and column detection

CODE CELL

```
def safe_float(x):
    if x is None:
        return np.nan
    if isinstance(x, (int, float, np.number)):
        return float(x)
    s = str(x).strip()
    if s in {"", ".", "nan", "None", "—", "-", "N/A", "NA"}:
        return np.nan
    m = re.search(r"[-+]?[d*]?[d+](?:[eE]([-+]?[d+])?", s.replace(",", ""))
    return float(m.group()) if m else np.nan
```

```
def normalize_formula(formula):
    if pd.isna(formula):
        return None
    s = str(formula).replace("_", "").replace("{", "").replace("}", "")
    s = s.replace(" ", "")
    try:
        return Composition(s).reduced_formula
    except Exception:
        return None
```

```
def elements_in_formula(formula):
    try:
        return {el.symbol for el in Composition(formula).elements}
    except Exception:
        return set()
```

```
def has_rare_earth(formula):
    return bool(elements_in_formula(formula) & RARE_EARTHS)
```

```
def has_actinide(formula):
    return bool(elements_in_formula(formula) & ACTINIDES)
```

```
def extract_spacegroup_number(x):
    if pd.isna(x):
        return np.nan
    s = str(x)
```

m = re.search(r"**ParseError: KaTeX parse error: Undefined control sequence: \d at position 2:**

\d{1,3}", s)

if m:

return int(m.group(1))

nums = re.findall(r"\b\d{1,3}\b", s)

nums = [int(n) for n in nums if 1 <= int(n) <= 230]

return nums[-1] if nums else np.nan

def normalize_header(s):

s = str(s).lower()

replacements = {

"μ": "mu", "₀": "0", "ₛ": "s", "ρ": "rho", "κ": "kappa",

"₋": "_", "₊": "+", "₌": "=", "₍": "(", "₎": ")", "³": "3", "²": "2",

"{": "{", "}": "}", "^": "^", "\": "\\", "n": "n"

}

for a, b in replacements.items():

s = s.replace(a, b)

s = re.sub(r"\s+", " ", s).strip()

return s

def flatten_columns(df):

out = df.copy()

if isinstance(out.columns, pd.MultiIndex):

new_cols = []

for tup in out.columns:

parts = []

for x in tup:

sx = str(x)

if sx.lower().startswith("unnamed"):

continue

if sx not in parts:

parts.append(sx)

new_cols.append(" | ".join(parts))

out.columns = new_cols

out.columns = [str(c).strip() for c in out.columns]

return out

def find_col(df, must=(), any_of=(), exclude=()):

Find the best column using normalized header keyword logic.

```

candidates = []
for c in df.columns:
    h = normalize_header(c)
    if any(e in h for e in exclude):
        continue
    if must and not all(k in h for k in must):
        continue
    if any_of and not any(k in h for k in any_of):
        continue
    score = sum(k in h for k in must) * 10 + sum(k in h for k in any_of)
    candidates.append((score, len(h), c))
if not candidates:
    return None
candidates.sort(key=lambda z: (-z[0], z[1]))
return candidates[0][2]

def extract_mp_id_from_text(x):
    m = re.search(r"\bmp-\d+\b", str(x), flags=re.I)
    return m.group(0).lower() if m else None

print("Helper functions ready.")

```

```

=====
=====
=====

```

MARKDOWN / SECTION

Part 3 — Securely enter the Materials Project API key

CODE CELL

```
MP_API_KEY = getpass.getpass("Paste Materials Project API key (hidden): ").strip()
assert len(MP_API_KEY) > 10, "API key appears too short."
print("API key received securely.")
```

=====

=====

=====

MARKDOWN / SECTION

Part 4 — Download broad Materials Project candidate data

CODE CELL

```
exclude_elements = sorted(RARE_EARTHS | (ACTINIDES if EXCLUDE_ACTINIDES else set()))

wanted_fields = [
    "material_id",
    "formula_pretty",
    "density",
    "energy_above_hull",
    "formation_energy_per_atom",
    "symmetry",
    "magnetic_ordering",
    "total_magnetization",
    "total_magnetization_normalized_formula_units",
    "total_magnetization_normalized_vol",
    "theoretical",
```

```

"database_IDs",
]

with MPRester(MP_API_KEY) as mpr:
    available = set(mpr.materials.summary.available_fields)
    fields = [f for f in wanted_fields if f in available]
    print("Fields requested:", fields)

    docs = mpr.materials.summary.search(
        energy_above_hull=(0, MP_EHULL_BROAD_MAX),
        total_magnetization_normalized_formula_units=(MP_MOMENT_FU_MIN, None),
        num_elements=MP_NUM_ELEMENTS,
        exclude_elements=exclude_elements,
        include_gnome=MP_INCLUDE_GNOME,
        fields=fields,
        chunk_size=1000,
    )

    print("MP documents downloaded:", len(docs))

```

```

=====
=====
=====

```

MARKDOWN / SECTION

Part 5 — Convert Materials Project records to a clean table

CODE CELL

```

def get_sym_attr(sym, attr):
    try:
        return getattr(sym, attr)
    
```

except Exception:

return None

mp_rows = []

for d in docs:

sym = getattr(d, "symmetry", None)

ordering = getattr(d, "magnetic_ordering", None)

ordering_str = str(ordering) if ordering is not None else None

if ordering_str and "." in ordering_str:

ordering_str = ordering_str.split(".")[1]

formula = getattr(d, "formula_pretty", None)

```
mp_rows.append({
    "mp_id": str(getattr(d, "material_id", "")),
    "formula_pretty": formula,
    "reduced_formula": normalize_formula(formula),
    "density_gcm3": safe_float(getattr(d, "density", np.nan)),
    "energy_above_hull_eVatom": safe_float(getattr(d, "energy_above_hull",
    "formation_energy_eVatom": safe_float(getattr(d, "formation_energy_per
    "spacegroup_symbol": get_sym_attr(sym, "symbol"),
    "spacegroup_number": safe_float(get_sym_attr(sym, "number")),
    "crystal_system": str(get_sym_attr(sym, "crystal_system")) if sym else
    "mp_magnetic_ordering": ordering_str,
    "mp_total_moment_muB_cell": safe_float(getattr(d, "total_magnetization
    "mp_moment_muB_fu": safe_float(getattr(d, "total_magnetization_normali
    "mp_moment_muB_A3": safe_float(getattr(d, "total_magnetization_normali
    "mp_theoretical": getattr(d, "theoretical", None),
})
```

mp = pd.DataFrame(mp_rows)

mp = mp.dropna(subset=["reduced_formula", "density_gcm3", "energy_above_hull_eVatom"])

mp = mp[~mp["reduced_formula"].map(has_rare_earth)].copy()

if EXCLUDE_ACTINIDES:

mp = mp[~mp["reduced_formula"].map(has_actinide)].copy()

mp["spacegroup_number"] = pd.to_numeric(

mp["spacegroup_number"], errors="coerce"

).astype("Int64")

mp["mp_ordering_flag"] = mp["mp_magnetic_ordering"].fillna("UNKNOWN")

mp.to_csv(OUT / "01_mp_candidates_broad.csv", index=False)

```
print("RE-free broad MP candidates:", len(mp))
display(mp.head())
print(mp["mp_ordering_flag"].value_counts(dropna=False).head(10))
```

```
=====
=====
=====
```

MARKDOWN / SECTION

Part 6 — Load NovoMag magnetic-property data

CODE CELL

```
NOVOMAG_URL = "https://magmat.herokuapp.com/"

try:
    tables = pd.read_html(NOVOMAG_URL)
    plausible = [t for t in tables if len(t) > 100 and t.shape[1] >= 10]
    if not plausible:
        raise RuntimeError("No large NovoMag table found.")
    novo_raw = flatten_columns(max(plausible, key=len))
    print("NovoMag table loaded:", novo_raw.shape)
    print("\nColumns:")
    for i, c in enumerate(novo_raw.columns):
        print(i, ":", c)
    except Exception as e:
        print("Automatic HTML import failed:", e)
        print("Fallback: upload magdb.json from Zenodo record 17399362.")
    from google.colab import files
    uploaded = files.upload()
    fn = next(iter(uploaded))
```

```
with open(fn, "r", encoding="utf-8") as f:
```

```
data = json.load(f)
```

```
if isinstance(data, list):
```

```
    records = data
```

```
elif isinstance(data, dict):
```

```
    list_values = [(k, v) for k, v in data.items() if isinstance(v, list)]
```

```
    if list_values:
```

```
        records = max(list_values, key=lambda kv: len(kv[1]))[1]
```

```
    elif all(isinstance(v, dict) for v in data.values()):
```

```
        records = list(data.values())
```

```
    else:
```

```
        raise ValueError("Could not identify a record list in magdb.json.")
```

```
else:
```

```
    raise ValueError("Unsupported JSON top-level structure.")
```

```
novo_raw = flatten_columns(pd.json_normalize(records))
```

```
print("JSON normalized:", novo_raw.shape)
```

```
print("\nColumns:")
```

```
for i, c in enumerate(novo_raw.columns):
```

```
    print(i, ":", c)
```

```
novo_raw.to_csv(OUT / "02_novomag_raw_table.csv", index=False)
```

```
=====
```

```
=====
```

```
=====
```

MARKDOWN / SECTION

Part 7 — Auto-detect and standardize NovoMag columns

CODE CELL

```
novo_cols = {
    "id": find_col(novo_raw, any_of=("materials id", "material id", "mmd")),
    "formula": find_col(novo_raw, must=("formula",), exclude=("formula units",)),
    "spacegroup": find_col(novo_raw, any_of=("space group", "spacegroup")),
    "crystal_system": find_col(novo_raw, any_of=("crystal system",)),
    "density": find_col(novo_raw, any_of=("density",)),
    "ehull": find_col(novo_raw, any_of=("convex hull", "relative to convex", "above hull")),
    "formation_energy": find_col(novo_raw, any_of=("formation energy",), exclude=("hull",)),
    "moment_atom": find_col(novo_raw, any_of=("averaged magnetic moment", "average magnetic moment")),
    "Js": find_col(novo_raw, any_of=("magnetic polarization", "j_s", "js")),
    "Tc": find_col(novo_raw, any_of=("curie temperature", "t_c", "tc")),
    "easy_axis": find_col(novo_raw, any_of=("magnetic easy axis", "easy axis")),
    "methods": find_col(novo_raw, any_of=("methods", "method")),
    "references": find_col(novo_raw, any_of=("references", "reference")),
}
```

```
def find_direction_col(direction):
    direction = direction.lower()
    for c in novo_raw.columns:
        h = normalize_header(c)
        if direction in h and ("anisotropy" in h or "mj/m" in h or "magnetic" in h):
            return c
    for c in novo_raw.columns:
        if direction in normalize_header(c):
            return c
    return None
```

```
novo_cols["K_ac"] = find_direction_col("a-c")
novo_cols["K_bc"] = find_direction_col("b-c")
novo_cols["K_ba"] = find_direction_col("b-a")
novo_cols["K_da"] = find_direction_col("d-a")
```

```
print("Detected NovoMag mapping:")
for k, v in novo_cols.items():
    print(f"{k:18s} -> {v}")
```

```

missing_basic = [k for k in ["formula", "Js"] if novo_cols.get(k) is None]
if missing_basic:
    raise ValueError(
        "Could not identify required NovoMag columns: "
        + ", ".join(missing_basic)
        + ". Inspect printed columns and edit novo_cols manually."
    )

novo = pd.DataFrame()
novo["novomag_id"] = (
    novo_raw[novo_cols["id"]].astype(str) if novo_cols["id"] else None
)
novo["formula_raw"] = novo_raw[novo_cols["formula"]]
novo["reduced_formula"] = novo["formula_raw"].map(normalize_formula)

if novo_cols["spacegroup"]:
    novo["spacegroup_raw"] = novo_raw[novo_cols["spacegroup"]]
    novo["spacegroup_number"] = novo["spacegroup_raw"].map(extract_spacegroup_number)
else:
    novo["spacegroup_raw"] = None
    novo["spacegroup_number"] = np.nan

novo["crystal_system_novo"] = (
    novo_raw[novo_cols["crystal_system"]] if novo_cols["crystal_system"] else None
)
novo["novo_density_gcm3"] = (
    novo_raw[novo_cols["density"]].map(safe_float) if novo_cols["density"] else np.nan
)
novo["novo_ehull_eVatom"] = (
    novo_raw[novo_cols["ehull"]].map(safe_float) if novo_cols["ehull"] else np.nan
)
novo["novo_formation_energy_eVatom"] = (
    novo_raw[novo_cols["formation_energy"]].map(safe_float)
    if novo_cols["formation_energy"] else np.nan
)
novo["moment_muB_atom_novo"] = (
    novo_raw[novo_cols["moment_atom"]].map(safe_float)
    if novo_cols["moment_atom"] else np.nan
)

```

```

novo["Js_T"] = novo_raw[novo_cols["Js"]].map(safe_float)
novo["Tc_K_novo"] = (
    novo_raw[novo_cols["Tc"]].map(safe_float) if novo_cols["Tc"] else np.nan
)
novo["easy_axis_novo"] = (
    novo_raw[novo_cols["easy_axis"]].map(safe_float) if novo_cols["easy_axis"] else None
)
novo["method_novo"] = (
    novo_raw[novo_cols["methods"]].map(safe_float) if novo_cols["methods"] else None
)
novo["reference_novo"] = (
    novo_raw[novo_cols["references"]].map(safe_float) if novo_cols["references"] else None
)

for name, key in [
    ("K_ac_MJm3", "K_ac"),
    ("K_bc_MJm3", "K_bc"),
    ("K_ba_MJm3", "K_ba"),
    ("K_da_MJm3", "K_da")
]:
    col = novo_cols.get(key)
    novo[name] = novo_raw[col].map(safe_float) if col else np.nan

novo["mp_id_from_novomag"] = (
    novo_raw.astype(str).agg(" ".join, axis=1).map(extract_mp_id_from_text)
)

novo["spacegroup_number"] = pd.to_numeric(
    novo["spacegroup_number"], errors="coerce"
).astype("Int64")

novo = novo.dropna(subset=["reduced_formula"]).copy()
novo = novo[~novo["reduced_formula"].map(has_rare_earth)].copy()
if EXCLUDE_ACTINIDES:
    novo = novo[~novo["reduced_formula"].map(has_actinide)].copy()

print("Standardized NovoMag records:", len(novo))
display(novo.head())
novo.to_csv(OUT / "03_novomag_standardized.csv", index=False)

```

=====

=====

=====

MARKDOWN / SECTION

Part 8 — Construct a directional anisotropy-barrier proxy

For available directional energy differences, reconstruct relative energies and use the energy span:

$$K_{\text{eff,barrier}} = E_{\text{max}} - E_{\text{min}}.$$

For cubic entries with K^{d-a} , use $|K^{d-a}|$.

This is a screening proxy; do not automatically relabel it conventional uniaxial K_1 .

CODE CELL

```
def anisotropy_barrier_MJm3(row):
    kac = row.get("K_ac_MJm3", np.nan)
    kbc = row.get("K_bc_MJm3", np.nan)
    kba = row.get("K_ba_MJm3", np.nan)
    kda = row.get("K_da_MJm3", np.nan)
```

```

if pd.notna(kda):
    return abs(float(kda))

energies = [0.0] # Ec reference
if pd.notna(kac):
    energies.append(float(kac)) # Ea - Ec
if pd.notna(kbc):
    energies.append(float(kbc)) # Eb - Ec

if len(energies) >= 2:
    return max(energies) - min(energies)

if pd.notna(kba):
    return abs(float(kba))

return np.nan

novo["Keff_barrier_MJm3"] = novo.apply(anisotropy_barrier_MJm3, axis=1)

novo["mag_property_completeness"] = (
novo["Js_T", "Tc_K_novo", "Keff_barrier_MJm3"]
.notna()
.sum(axis=1)
)

print(novo["Js_T", "Tc_K_novo", "Keff_barrier_MJm3"].describe())

```

```

=====

=====

=====

```

MARKDOWN / SECTION

Part 9 — Load NEMAD Curie-temperature data as a formula-level supplement

CODE CELL

```
NEMAD_TC_URL = (  
    "https://raw.githubusercontent.com/sumanitani/"  
    "NEMAD-MagneticML/main/Dataset/FM_with_curie.csv"  
)  
  
try:  
    nemad_raw = pd.read_csv(NEMAD_TC_URL)  
    print("NEMAD TC dataset:", nemad_raw.shape)  
    print("Columns:", list(nemad_raw.columns))  
except Exception as e:  
    print("Automatic NEMAD GitHub download failed:", e)  
    from google.colab import files  
    print("Upload FM_with_curie.csv from the NEMAD-MagneticML repository.")  
    uploaded = files.upload()  
    fn = next(iter(uploaded))  
    nemad_raw = pd.read_csv(fn)  
  
formula_col = find_col(  
    nemad_raw,  
    any_of=("composition", "formula", "material", "compound")  
)  
tc_col = find_col(  
    nemad_raw,  
    any_of=("mean_tc_k", "mean tc", "curie", "tc")  
)  
  
print("NEMAD formula column:", formula_col)  
print("NEMAD TC column:", tc_col)  
  
if formula_col is None or tc_col is None:  
    raise ValueError("Could not auto-detect NEMAD formula or TC column.")  
  
nemad_tc = pd.DataFrame({  
    "reduced_formula": nemad_raw[formula_col].map(normalize_formula),  
    "Tc_K_nemad": nemad_raw[tc_col].map(safe_float),  
})
```

```

nemad_tc = nemad_tc.dropna(subset=["reduced_formula", "Tc_K_nemad"])
nemad_tc = nemad_tc[~nemad_tc["reduced_formula"].map(has_rare_earth)]

nemad_tc_agg = (
nemad_tc.groupby("reduced_formula", as_index=False)
.agg(
Tc_K_nemad=("Tc_K_nemad", "median"),
nemad_tc_record_count=("Tc_K_nemad", "size")
)
)

nemad_tc_agg.to_csv(OUT / "04_nemad_tc_formula_level.csv", index=False)
print("Unique NEMAD FM formulas with TC:", len(nemad_tc_agg))
display(nemad_tc_agg.head())

```

```
=====
```

```
=====
```

```
=====
```

MARKDOWN / SECTION

Part 10 — Cross-match Materials Project ↔ NovoMag

CODE CELL

```

novo_sorted = novo.sort_values(
["mag_property_completeness", "Keff_barrier_MJm3", "Js_T"],
ascending=[False, False, False],
na_position="last"
).copy()

by_mpid = {}
for _, r in novo_sorted.dropna(subset=["mp_id_from_novomag"]).iterrows():
by_mpid.setdefault(str(r["mp_id_from_novomag"]), r)

```

```
by_formula_sg = {}
for _, r in novo_sorted.dropna(
    subset=["reduced_formula", "spacegroup_number"]
).iterrows():
    key = (r["reduced_formula"], int(r["spacegroup_number"]))
    by_formula_sg.setdefault(key, r)

formula_groups = {
    f: g.copy() for f, g in novo_sorted.groupby("reduced_formula")
}

selected_rows = []

for _, m in mp.iterrows():
    chosen = None
    quality = "no_novomag_match"
```

```

if m["mp_id"] in by_mpid:
    chosen = by_mpid[m["mp_id"]]
    quality = "direct_mp_id"

if chosen is None and pd.notna(m["spacegroup_number"]):
    key = (m["reduced_formula"], int(m["spacegroup_number"]))
    if key in by_formula_sg:
        chosen = by_formula_sg[key]
        quality = "formula_plus_spacegroup"

if (
    chosen is None
    and ALLOW_UNIQUE_FORMULA_ONLY_NOVOMAG
    and m["reduced_formula"] in formula_groups
):
    g = formula_groups[m["reduced_formula"]]
    if len(g) == 1:
        chosen = g.iloc[0]
        quality = "unique_formula_only"

base = m.to_dict()
base["novomag_match_quality"] = quality

if chosen is not None:
    for col in novo.columns:
        if col != "reduced_formula":
            base[col] = chosen[col]

selected_rows.append(base)

merged = pd.DataFrame(selected_rows)

print("Cross-match counts:")
print(merged["novomag_match_quality"].value_counts())

merged.to_csv(OUT / "05_mp_novomag_crossmatch_all.csv", index=False)

mag = merged[
    merged["novomag_match_quality"] != "no_novomag_match"
].copy()

print("MP candidates with a NovoMag match:", len(mag))

```

=====

=====

=====

MARKDOWN / SECTION

Part 11 — Supplement missing Curie temperatures with NEMAD and preserve provenance

CODE CELL

```
mag = mag.merge(nemad_tc_agg, on="reduced_formula", how="left")

mag["Tc_K"] = mag["Tc_K_novo"]
mag["Tc_source"] = np.where(
mag["Tc_K_novo"].notna(),
"NovoMag",
None
)

fill_tc = mag["Tc_K"].isna() & mag["Tc_K_nemad"].notna()
mag.loc[fill_tc, "Tc_K"] = mag.loc[fill_tc, "Tc_K_nemad"]
mag.loc[fill_tc, "Tc_source"] = "NEMAD_formula_level"

print("TC source counts:")
print(mag["Tc_source"].value_counts(dropna=False))
```

=====

=====

=====

MARKDOWN / SECTION

Part 12 — Optional curated literature overlay

CODE CELL

```
lit_template = pd.DataFrame(columns=[
    "mp_id",
    "formula",
    "spacegroup_number",
    "Js_T",
    "Tc_K",
    "Keff_MJm3",
    "Br_T",
    "Hc_kAm",
    "source_DOI",
    "data_type",
    "notes"
])
lit_template.to_csv(OUT / "literature_property_template.csv", index=False)

USE_LITERATURE_OVERLAY = False # change to True after filling the template

if USE_LITERATURE_OVERLAY:
    from google.colab import files
    uploaded = files.upload()
    lit_fn = next(iter(uploaded))
    lit = pd.read_csv(lit_fn)
```

```

lit["reduced_formula"] = lit["formula"].map(normalize_formula)
lit["spacegroup_number"] = pd.to_numeric(
    lit["spacegroup_number"], errors="coerce"
).astype("Int64")

for c in ["Js_T", "Tc_K", "Keff_MJm3", "Br_T", "Hc_kAm"]:
    lit[c] = pd.to_numeric(lit[c], errors="coerce")

def best_lit_match(row):
    q = lit[lit["mp_id"].astype(str) == str(row["mp_id"])]
    if len(q):
        return q.iloc[0], "literature_mp_id"

    if pd.isna(row["spacegroup_number"]):
        q = lit[
            (lit["reduced_formula"] == row["reduced_formula"]) &
            (lit["spacegroup_number"] == int(row["spacegroup_number"]))
        ]
        if len(q):
            return q.iloc[0], "literature_formula_sg"

    q = lit[lit["reduced_formula"] == row["reduced_formula"]]
    if len(q) == 1:
        return q.iloc[0], "literature_formula_only"

    return None, None

mag["literature_match_quality"] = None
mag["source_D0I"] = None
mag["literature_data_type"] = None
mag["Br_T"] = np.nan
mag["Hc_kAm"] = np.nan
mag["Js_source"] = "NovoMag"
mag["K_source"] = "NovoMag_directional_barrier"

for idx, row in mag.iterrows():
    lr, q = best_lit_match(row)
    if lr is None:
        continue

    mag.at[idx, "literature_match_quality"] = q
    mag.at[idx, "source_D0I"] = lr.get("source_D0I", None)
    mag.at[idx, "literature_data_type"] = lr.get("data_type", None)

```

```

mag.at[idx, "Br_T"] = lr.get("Br_T", np.nan)
mag.at[idx, "Hc_kAm"] = lr.get("Hc_kAm", np.nan)

if pd.notna(lr.get("Js_T", np.nan)):
    mag.at[idx, "Js_T"] = lr["Js_T"]
    mag.at[idx, "Js_source"] = f"literature:{lr.get('source_DOI', '')}"
if pd.notna(lr.get("Tc_K", np.nan)):
    mag.at[idx, "Tc_K"] = lr["Tc_K"]
    mag.at[idx, "Tc_source"] = f"literature:{lr.get('source_DOI', '')}"
if pd.notna(lr.get("Keff_MJm3", np.nan)):
    mag.at[idx, "Keff_barrier_MJm3"] = lr["Keff_MJm3"]
    mag.at[idx, "K_source"] = f"literature:{lr.get('source_DOI', '')}"

else:
mag["literature_match_quality"] = None
mag["source_DOI"] = None
mag["literature_data_type"] = None
mag["Br_T"] = np.nan
mag["Hc_kAm"] = np.nan
mag["Js_source"] = "NovoMag"
mag["K_source"] = "NovoMag_directional_barrier"

print("Literature template saved to:", OUT / "literature_property_template.csv")

```

```

=====
=====
=====

```

MARKDOWN / SECTION

Part 13 — Calculate lightweight hard-magnet descriptors

$$M_s = J_s / \mu_0$$

$$M_{s,\text{specific}} = M_s / \rho$$

$$\kappa = \sqrt{\frac{K_{\text{eff}}}{\mu_0 M_s^2}}$$

Ideal intrinsic estimate:

$$(BH)_{\text{max}}^{\text{ideal}} \approx J_s^2 / (4\mu_0)$$

and gravimetric form:

$$(BH)_{\text{max,grav}}^{\text{ideal}} = (BH)_{\text{max}}^{\text{ideal}} / \rho.$$

CODE CELL

```
for c in [
    "Js_T", "Tc_K", "Keff_barrier_MJm3",
    "density_gcm3", "energy_above_hull_eVatom", "Br_T"
]:
    mag[c] = pd.to_numeric(mag[c], errors="coerce")

mag["density_kgm3"] = mag["density_gcm3"] * 1000.0
mag["Ms_Apm"] = mag["Js_T"] / MU0
mag["Ms_specific_Am2kg"] = (
    mag["Ms_Apm"] / mag["density_kgm3"]
)

mag["Keff_Jm3"] = mag["Keff_barrier_MJm3"] * 1e6
mag["kappa_calc"] = np.sqrt(
    mag["Keff_Jm3"] /
    (MU0 * mag["Ms_Apm"]**2)
)

mag["BHmax_ideal_Jm3_from_Js"] = (
    mag["Js_T"]**2 / (4.0 * MU0)
)
mag["BHmax_ideal_kJm3_from_Js"] = (
    mag["BHmax_ideal_Jm3_from_Js"] / 1000.0
)
mag["BHmax_ideal_grav_Jkg"] = (
```

```

mag["BHmax_ideal_Jm3_from_Js"] /
mag["density_kgm3"]
)

mag["BHmax_ideal_Jm3_from_Br"] = np.where(
mag["Br_T"].notna(),
mag["Br_T"]**2 / (4.0 * MU0),
np.nan
)

mag["BHmax_ideal_grav_Jkg_from_Br"] = (
mag["BHmax_ideal_Jm3_from_Br"] /
mag["density_kgm3"]
)

mag["density_rel_diff"] = np.where(
mag["novo_density_gcm3"].notna() &
(mag["density_gcm3"] > 0),
np.abs(
mag["novo_density_gcm3"] -
mag["density_gcm3"]
) / mag["density_gcm3"],
np.nan
)

display(
mag["mp_id", "reduced_formula", "density_gcm3", "Js_T", "Tc_K", "Keff_barrier_MJm3",
"kappa_calc", "Ms_specific_Am2kg", "BHmax_ideal_kJm3_from_Js", "BHmax_ideal_grav_Jkg"].head()
)

```

```
=====
```

```
=====
```

=====

MARKDOWN / SECTION

Part 14 — Assign data-confidence grades

CODE CELL

```
def confidence_grade(row):  
    complete = all(  
        pd.notna(row.get(c, np.nan))  
        for c in ["Js_T", "Tc_K", "Keff_barrier_MJm3"]  
    )
```

```

if not complete:
    return "D"

lit_q = row.get("literature_match_quality", None)
novo_q = row.get("novomag_match_quality", None)
tc_src = str(row.get("Tc_source", ""))

```

```

if lit_q == "literature_mp_id":
    return "A"

```

```

if (
    novo_q == "direct_mp_id"
    and "NEMAD_formula_level" not in tc_src
):
    return "A"

```

```

if lit_q == "literature_formula_sg":
    return "B"

```

```

if (
    novo_q == "formula_plus_spacegroup"
    and "NEMAD_formula_level" not in tc_src
):
    return "B"

```

```

if (
    novo_q == "unique_formula_only"
    or "NEMAD_formula_level" in tc_src
    or lit_q == "literature_formula_only"
):
    return "C"

```

```

return "D"

```

```

mag["data_confidence"] = mag.apply(confidence_grade, axis=1)
mag["confidence_rank"] = mag["data_confidence"].map(
{"A":1, "B":2, "C":3, "D":4}
)

```

```

print(mag["data_confidence"].value_counts(dropna=False))
mag.to_csv(OUT / "06_crossmatched_with_descriptors.csv", index=False)

```

=====

=====

=====

MARKDOWN / SECTION

Part 15 — Apply minimum physical requirements

CODE CELL

```
complete_mask = (  
mag["Js_T"].notna() &  
mag["Tc_K"].notna() &  
mag["Keff_barrier_MJm3"].notna() &  
mag["kappa_calc"].notna() &  
mag["density_gcm3"].notna() &  
mag["energy_above_hull_eVatom"].notna()  
)  
  
physical_mask = (  
complete_mask &  
(mag["energy_above_hull_eVatom"] <= PRIMARY_EHULL_MAX) &  
(mag["Tc_K"] >= PRIMARY_TC_MIN) &  
(mag["Js_T"] >= PRIMARY_JS_MIN) &  
(mag["Keff_barrier_MJm3"] >= PRIMARY_KEFF_MIN) &  
(mag["kappa_calc"] >= PRIMARY_KAPPA_MIN)  
)  
  
qualified = mag[physical_mask].copy()  
  
print("Cross-matched magnetic candidates:", len(mag))  
print(  
"Complete Js + TC + anisotropy + density + Ehull:",  
int(complete_mask.sum())
```

```

)
print("Pass primary physical requirements:", len(qualified))

qualified.to_csv(OUT / "07_primary_physical_screen.csv", index=False)

display(
qualified["mp_id", "reduced_formula", "data_confidence",
"density_gcm3", "energy_above_hull_eVatom", "Js_T", "Tc_K", "Keff_barrier_MJm3",
"kappa_calc", "Ms_specific_Am2kg", "BHmax_ideal_grav_Jkg"
.sort_values("BHmax_ideal_grav_Jkg", ascending=False)
.head(20)
)

```

```

=====
=====
=====

```

MARKDOWN / SECTION

Part 16 — Multiobjective Pareto optimization

CODE CELL

```

def pareto_mask(df, maximize, minimize):
work = df.dropna(subset=maximize + minimize).copy()

```

```

if work.empty:
    return pd.Series(False, index=df.index)

Xmax = work[maximize].to_numpy(dtype=float)
Xmin = -work[minimize].to_numpy(dtype=float)
X = np.hstack([Xmax, Xmin])

efficient = np.ones(len(work), dtype=bool)

for i in range(len(work)):
    ge_all = np.all(X >= X[i], axis=1)
    gt_any = np.any(X > X[i], axis=1)
    efficient[i] = not np.any(ge_all & gt_any)

result = pd.Series(False, index=df.index)
result.loc[work.index] = efficient
return result

```

```

MAXIMIZE = [
    "Ms_specific_Am2kg",
    "BHmax_ideal_grav_Jkg",
    "Tc_K",
    "kappa_calc",
]

```

```

MINIMIZE = [
    "density_gcm3",
    "energy_above_hull_eVatom",
]

```

```

qualified["is_pareto_primary"] = pareto_mask(
    qualified,
    MAXIMIZE,
    MINIMIZE
)

```

```

pareto = qualified[
    qualified["is_pareto_primary"]
].copy()

```

```

pareto.to_csv(
OUT / "08_primary_pareto_front.csv",
index=False
)

print("Primary physical-screen candidates:", len(qualified))
print("Primary Pareto-front candidates:", len(pareto))

display(
pareto["mp_id", "reduced_formula", "data_confidence", "density_gcm3", "energy_above_hull_eVatom",
"Js_T", "Tc_K", "Keff_barrier_MJm3", "kappa_calc", "Ms_specific_Am2kg", "BHmax_ideal_grav_Jkg"
].sort_values(
["data_confidence", "BHmax_ideal_grav_Jkg"],
ascending=[True, False]
)
)

```

```

=====
=====
=====

```

MARKDOWN / SECTION

Part 17 — Sensitivity analysis

CODE CELL

```

EHULL_GRID = [0.05, 0.10, 0.20]
TC_GRID = [300.0, 400.0, 500.0]
JS_GRID = [0.50, 0.80, 1.00]
KAPPA_GRID = [0.80, 1.00, 1.20]

mag["sensitivity_qualified_count"] = 0
mag["sensitivity_pareto_count"] = 0

```

```
sens_rows = []  
n_settings = 0  
  
for eh, tc, js, kap in itertools.product(  
    EHULL_GRID,  
    TC_GRID,  
    JS_GRID,  
    KAPPA_GRID  
):  
    n_settings += 1
```

```

mask = (
    complete_mask &
    (mag["energy_above_hull_eVatom"] <= eh) &
    (mag["Tc_K"] >= tc) &
    (mag["Js_T"] >= js) &
    (mag["Keff_barrier_MJm3"] >= PRIMARY_KEFF_MIN) &
    (mag["kappa_calc"] >= kap)
)

```

```

sub = mag[mask].copy()

```

```

mag.loc[
    sub.index,
    "sensitivity_qualified_count"
] += 1

```

```

if len(sub):
    pmask = pareto_mask(sub, MAXIMIZE, MINIMIZE)
    pidx = sub.index[pmask]
    mag.loc[
        pidx,
        "sensitivity_pareto_count"
    ] += 1
    n_pareto = int(pmask.sum())
else:
    n_pareto = 0

```

```

sens_rows.append({
    "Ehull_max_eVatom": eh,
    "Tc_min_K": tc,
    "Js_min_T": js,
    "kappa_min": kap,
    "n_qualified": len(sub),
    "n_pareto": n_pareto,
})

```

```

sens = pd.DataFrame(sens_rows)

```

```

mag["sensitivity_qualified_fraction"] = (
    mag["sensitivity_qualified_count"] / n_settings
)
mag["sensitivity_pareto_fraction"] = (

```

```
mag["sensitivity_pareto_count"] / n_settings
)

sens.to_csv(
OUT / "09_threshold_sensitivity_summary.csv",
index=False
)

mag.to_csv(
OUT / "10_all_candidates_with_robustness.csv",
index=False
)

print("Threshold combinations tested:", n_settings)
display(sens.head(12))
```

=====

=====

=====

MARKDOWN / SECTION

Part 18 — Build a transparent 5–15 candidate literature-check shortlist

CODE CELL

```
rob_cols = [
"mp_id",
"sensitivity_qualified_count",
"sensitivity_pareto_count",
"sensitivity_qualified_fraction",
"sensitivity_pareto_fraction"
]
```

```
pareto2 = pareto.drop(
columns=[
c for c in rob_cols[1:]
if c in pareto.columns
],
errors="ignore"
)
```

```
pareto2 = pareto2.merge(
mag[rob_cols],
on="mp_id",
how="left"
)
```

```
shortlist_pool = pareto2[
pareto2["data_confidence"].isin(
SHORTLIST_CONFIDENCE
)
].copy()
```

```
shortlist = shortlist_pool.sort_values(
[
"sensitivity_pareto_count",
"confidence_rank",
"energy_above_hull_eVatom",
"BHmax_ideal_grav_Jkg",
],
ascending=[False, True, True, False]
).head(SHORTLIST_N)
```

```
shortlist.to_csv(
OUT / "11_literature_check_shortlist.csv",
index=False
)
```

```
cols = [
"mp_id", "reduced_formula",
"spacegroup_symbol", "spacegroup_number",
"data_confidence", "novomag_match_quality",
"Tc_source",
```

```
"density_gcm3","energy_above_hull_eVatom",  
"Js_T","Tc_K","Keff_barrier_MJm3",  
"kappa_calc","Ms_specific_Am2kg",  
"BHmax_ideal_kJm3_from_Js",  
"BHmax_ideal_grav_Jkg",  
"sensitivity_pareto_count",  
"source_DOI"  
]
```

```
display(  
shortlist[  
[c for c in cols if c in shortlist.columns]  
]  
)
```

```
print("Shortlist size:", len(shortlist))
```

```
=====
```

```
=====
```

```
=====
```

MARKDOWN / SECTION

Part 19 — Data-quality diagnostics

CODE CELL

```
quality_summary = pd.DataFrame({  
"metric": [  
"Broad MP candidates",  
"NovoMag-matched candidates",  
"Complete Js+TC+K+density+Ehull",  
"Pass primary physical screen",
```

```

"Primary Pareto candidates",
"Final literature-check shortlist",
"Direct MP-ID NovoMag matches",
"Formula+space-group NovoMag matches",
"Unique-formula-only NovoMag matches",
"NEMAD formula-level TC supplements",
"Density mismatch >10% (MP vs NovoMag)"
],
"value": [
len(mp),
len(mag),
int(complete_mask.sum()),
len(qualified),
len(pareto),
len(shortlist),
int(
(
mag["novomag_match_quality"] ==
"direct_mp_id"
).sum()
),
int(
(
mag["novomag_match_quality"] ==
"formula_plus_spacegroup"
).sum()
),
int(
(
mag["novomag_match_quality"] ==
"unique_formula_only"
).sum()
),
int(
(
mag["Tc_source"] ==
"NEMAD_formula_level"
).sum()
)
]

```

```

),
int(
(
mag["density_rel_diff"] > 0.10
).sum()
),
]
})

quality_summary.to_csv(
OUT / "12_data_quality_summary.csv",
index=False
)

display(quality_summary)

warnings_df = mag[
(mag["density_rel_diff"] > 0.10) |
(mag["data_confidence"] == "C")
].copy()

warnings_df.to_csv(
OUT / "13_manual_review_flags.csv",
index=False
)

print("Rows flagged for manual review:", len(warnings_df))

```

=====

=====

=====

MARKDOWN / SECTION

Part 20 — Figure: density vs magnetic polarization

CODE CELL

```
plot_df = mag.dropna(
subset=["density_gcm3", "Js_T"]
).copy()

fig, ax = plt.subplots(figsize=(7.2, 5.5))

ax.scatter(
plot_df["density_gcm3"],
plot_df["Js_T"],
s=28,
alpha=0.7
)

if len(shortlist):
ax.scatter(
shortlist["density_gcm3"],
shortlist["Js_T"],
s=80,
marker="*",
label="Literature-check shortlist"
)

ax.set_xlabel(
r"Density (g cm-3)",
fontsize=12
)
ax.set_ylabel(
r"Magnetic polarization  $J_s$  (T)",
fontsize=12
)
ax.tick_params(labelsize=11)

for spine in ax.spines.values():
spine.set_visible(True)

if len(shortlist):
ax.legend(frameon=False)
```

```
fig.tight_layout()
fig.savefig(
OUT / "Fig_density_vs_Js.png",
dpi=600,
bbox_inches="tight"
)
plt.show()
```

```
=====
=====
=====
```

MARKDOWN / SECTION

Part 21 — Figure: specific magnetization vs Curie temperature

CODE CELL

```
plot_df = mag.dropna(
subset=["Ms_specific_Am2kg", "Tc_K"]
).copy()

fig, ax = plt.subplots(figsize=(7.2, 5.5))

ax.scatter(
plot_df["Ms_specific_Am2kg"],
plot_df["Tc_K"],
s=28,
alpha=0.7
)
```

```

if len(shortlist):
    ax.scatter(
        shortlist["Ms_specific_Am2kg"],
        shortlist["Tc_K"],
        s=80,
        marker="*",
        label="Literature-check shortlist"
    )

    ax.axhline(
        PRIMARY_TC_MIN,
        linestyle="--",
        linewidth=1
    )

    ax.set_xlabel(
        r"Specific magnetization  $M_s/\rho$  "
        r"(A m2 kg-1)",
        fontsize=12
    )
    ax.set_ylabel(
        r"Curie temperature  $T_C$  (K)",
        fontsize=12
    )
    ax.tick_params(labelsize=11)

for spine in ax.spines.values():
    spine.set_visible(True)

if len(shortlist):
    ax.legend(frameon=False)

fig.tight_layout()
fig.savefig(
    OUT / "Fig_specific_Ms_vs_Tc.png",
    dpi=600,
    bbox_inches="tight"
)
plt.show()

```

=====

=====

=====

MARKDOWN / SECTION

Part 22 — Figure: hardness parameter vs gravimetric ideal energy product

CODE CELL

```
plot_df = mag.dropna(
subset=[
    "kappa_calc",
    "BHmax_ideal_grav_Jkg"
]
).copy()

fig, ax = plt.subplots(figsize=(7.2, 5.5))

ax.scatter(
    plot_df["kappa_calc"],
    plot_df["BHmax_ideal_grav_Jkg"],
    s=28,
    alpha=0.7
)

if len(shortlist):
    ax.scatter(
        shortlist["kappa_calc"],
        shortlist["BHmax_ideal_grav_Jkg"],
        s=80,
        marker="*",
```

```
label="Literature-check shortlist"
```

```
)
```

```
ax.axvline(
```

```
PRIMARY_KAPPA_MIN,
```

```
linestyle="--",
```

```
linewidth=1
```

```
)
```

```
ax.set_xlabel(
```

```
r"Hardness parameter  $\kappa$ ",
```

```
fontsize=12
```

```
)
```

```
ax.set_ylabel(
```

```
r"Ideal gravimetric  $(BH)_{\max}$  "
```

```
r"(J kg-1)",
```

```
fontsize=12
```

```
)
```

```
ax.tick_params(labelsize=11)
```

```
for spine in ax.spines.values():
```

```
spine.set_visible(True)
```

```
if len(shortlist):
```

```
ax.legend(frameon=False)
```

```
fig.tight_layout()
```

```
fig.savefig(
```

```
OUT / "Fig_kappa_vs_BHmax_grav.png",
```

```
dpi=600,
```

```
bbox_inches="tight"
```

```
)
```

```
plt.show()
```

=====

=====

=====

MARKDOWN / SECTION

Part 23 — Figure: stability vs gravimetric ideal energy product

CODE CELL

```
plot_df = mag.dropna(
subset=[
"energy_above_hull_eVatom",
"BHmax_ideal_grav_Jkg"
]
).copy()

fig, ax = plt.subplots(figsize=(7.2, 5.5))

ax.scatter(
plot_df["energy_above_hull_eVatom"],
plot_df["BHmax_ideal_grav_Jkg"],
s=28,
alpha=0.7
)

if len(shortlist):
ax.scatter(
shortlist["energy_above_hull_eVatom"],
shortlist["BHmax_ideal_grav_Jkg"],
s=80,
marker="*",
label="Literature-check shortlist"
)

ax.axvline(
PRIMARY_EHULL_MAX,
```

```

linestyle="--",
linewidth=1
)

ax.set_xlabel(
r"Energy above hull (eV atom-1)",
fontsize=12
)
ax.set_ylabel(
r"Ideal gravimetric ( $BH$ )max "
r"(J kg-1)",
fontsize=12
)
ax.tick_params(labelsize=11)

for spine in ax.spines.values():
    spine.set_visible(True)

if len(shortlist):
    ax.legend(frameon=False)

fig.tight_layout()
fig.savefig(
OUT / "Fig_Ehull_vs_BHmax_grav.png",
dpi=600,
bbox_inches="tight"
)
plt.show()

```

```

=====

=====

```

=====

MARKDOWN / SECTION

Part 24 — 3D Pareto visualization

CODE CELL

```
from mpl_toolkits.mplot3d import Axes3D # noqa: F401
```

```
plot_df = qualified.dropna(  
subset=[  
"density_gcm3",  
"Tc_K",  
"BHmax_ideal_grav_Jkg"  
]  
)  
.copy()
```

```
fig = plt.figure(figsize=(8.0, 6.5))  
ax = fig.add_subplot(111, projection="3d")
```

```
ax.scatter(  
plot_df["density_gcm3"],  
plot_df["Tc_K"],  
plot_df["BHmax_ideal_grav_Jkg"],  
s=24,  
alpha=0.55  
)
```

```
if len(pareto):  
ax.scatter(  
pareto["density_gcm3"],  
pareto["Tc_K"],  
pareto["BHmax_ideal_grav_Jkg"],  
s=65,  
marker="^",
```

```

label="Primary Pareto front"
)

ax.set_xlabel(r"Density (g cm-3)")
ax.set_ylabel(r" $T_C$  (K)")
ax.set_zlabel(
r"Ideal gravimetric ( $BH$ )max "
r"(J kg-1)"
)

if len(pareto):
ax.legend(frameon=False)

fig.tight_layout()
fig.savefig(
OUT / "Fig_3D_Pareto.png",
dpi=600,
bbox_inches="tight"
)
plt.show()

```

=====

=====

=====

MARKDOWN / SECTION

Part 25 — Chemical-family and structure-family summaries

CODE CELL

```
def element_signature(formula):  
    els = sorted(elements_in_formula(formula))  
    return "-".join(els)
```

```
if len(pareto2):  
    pareto2["chemical_system"] = (  
        pareto2["reduced_formula"]  
        .map(element_signature)  
    )
```

```

chem_summary = (
    pareto2
    .groupby("chemical_system")
    .agg(
        n_candidates=("mp_id", "size"),
        median_density=("density_gcm3", "median"),
        median_Tc=("Tc_K", "median"),
        median_kappa=("kappa_calc", "median"),
        median_BHgrav=("BHmax_ideal_grav_Jkg", "median"),
    )
    .sort_values(
        "n_candidates",
        ascending=False
    )
    .reset_index()
)

```

```

struct_summary = (
    pareto2
    .groupby(
        [
            "crystal_system",
            "spacegroup_symbol"
        ],
        dropna=False
    )
    .agg(
        n_candidates=("mp_id", "size"),
        median_density=("density_gcm3", "median"),
        median_kappa=("kappa_calc", "median"),
        median_BHgrav=("BHmax_ideal_grav_Jkg", "median"),
    )
    .sort_values(
        "n_candidates",
        ascending=False
    )
    .reset_index()
)

```

```

chem_summary.to_csv(
    OUT / "14_pareto_chemical_families.csv",
    index=False
)

```

```

struct_summary.to_csv(
    OUT / "15_pareto_structure_families.csv",
    index=False
)

print("Chemical systems:")
display(chem_summary.head(20))

print("Structure families:")
display(struct_summary.head(20))

else:
print(
    "No primary Pareto candidates "
    "available for family summaries."
)

```

```

=====
=====
=====

```

MARKDOWN / SECTION

Part 26 — Benchmark template for established RE-free magnets

CODE CELL

```

benchmark_template = pd.DataFrame(columns=[
    "family_or_material",
    "formula",
    "phase_or_structure",
    "density_gcm3",

```

```
"Js_or_Br_T",
"Tc_K",
"K_or_Keff_MJm3",
"kappa",
"BHmax_measured_kJm3",
"BHmax_ideal_kJm3",
"BHmax_grav_Jkg",
"measurement_temperature_K",
"experimental_or_DFT",
"source_DOI",
"notes"
])

benchmark_template.to_csv(
OUT / "benchmark_RE_free_magnets_template.csv",
index=False
)

print(
"Benchmark template saved. "
"Fill it only from verified original sources."
)
```

=====

=====

=====

MARKDOWN / SECTION

Part 27 — Export final literature-verification sheet

CODE CELL

```
literature_check = shortlist.copy()

for c in [
    "verified_phase",
    "verified_magnetic_order",
    "verified_Tc_source",
    "verified_anisotropy_source",
    "verified_magnetization_source",
    "verified_synthesis_evidence",
    "verified_DOI_1",
    "verified_DOI_2",
    "manual_decision",
    "manual_notes",
]:
    literature_check[c] = ""

literature_check.to_excel(
    OUT / "FINAL_literature_verification_sheet.xlsx",
    index=False
)

literature_check.to_csv(
    OUT / "FINAL_literature_verification_sheet.csv",
    index=False
)

print(
    "Final verification sheet created:",
    len(literature_check),
    "candidates"
)
```

=====

=====

=====

MARKDOWN / SECTION

Part 28 — Package all results and download one ZIP

CODE CELL

```
import shutil

zip_base = "/content/Nur_Lightweight_RE_Free_PM_Results"
zip_path = shutil.make_archive(
zip_base,
"zip",
OUT
)

print("Created:", zip_path)
print("\nFiles:")

for p in sorted(OUT.iterdir()):
    print(" -", p.name)

from google.colab import files
files.download(zip_path)
```

=====

=====

=====

MARKDOWN / SECTION

Interpretation limits for the manuscript

1. Materials Project total magnetization is a broad calculated pre-screen, not final measured M_s .
2. Formula-only matching is not equivalent to phase-specific matching.
3. NEMAD T_C supplementation is formula-level unless structure identity is independently verified.
4. $K_{eff_barrier_MJm3}$ is a directional anisotropy-barrier proxy reconstructed from available directional energy differences; do not silently call it K_1 .
5. $\kappa \geq 1$ is an intrinsic hardness criterion; it does not guarantee practical coercivity.
6. $(BH)_{max}^{ideal}$ assumes ideal remanence/loop behavior and is not measured practical $(BH)_{max}$.
7. The final 5–15 candidates must be checked against original literature before publication.